

Descriptores Moleculares · De estructuras a números

Un descriptor molecular es cualquier valor numérico que captura alguna propiedad de una molécula. Son el puente entre la química y el machine learning.

Molécula (SMILES) → función matemática → número

CC(=O)Oc1ccccc1C(=O)O → MW = 180.04 · LogP = 1.19 · TPSA = 63.6 Å²

Constitucionales

Composición atómica:
conteos de átomos, fórmula
molecular, peso molecular.

Ejemplos:

MW, nC, nO, HeavyAtomCount

¿Para qué sirve?

Determinar drug-likeness básico



Fisicoquímicos

Propiedades medibles:
solubilidad, permeabilidad,
lipofilia, ionización.

Ejemplos:

LogP, TPSA, pKa, HBD, HBA

¿Para qué sirve?

Predecir ADME y biodisponibilidad



Topológicos

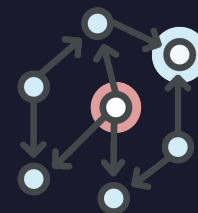
Forma del grafo molecular:
ramificación, conectividad,
anillos, caminos.

Ejemplos:

Chi, Kappa, BertzCT, Wiener

¿Para qué sirve?

Comparar scaffolds, clustering



3D / Geométricos

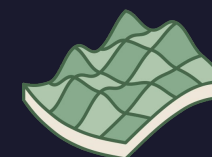
Coordenadas espaciales:
volumen, forma, superficie,
momentos de inercia.

Ejemplos:

PMI, ASA, WHIM, RDF

¿Para qué sirve?

Docking y farmacoforos 3D



Mordred · 1826 descriptores desde una línea de código

Librería open-source de Python (Moriwaki et al. 2018) que calcula 1826 descriptores 2D y 3D directamente desde SMILES con RDKit.

Módulos principales (1826 descriptores)

Autocorrelation

606
desc. Distribución electrónica / Topología

EState

316
desc. Estado electrónico / Reactividad local

MoRSE

160
desc. Distribución espacial 3D

RingCount

138
desc. Sistemas de anillos

BaryszMatrix

104
desc. Topología ponderada por átomos

Chi / Kappa

56 desc. Conectividad / Ramificación

MoeType

53 desc. Área de superficie accesible

CPSA

43 desc. Distribución de carga (3D)

Otros módulos

350
desc. Carga, entropía, distancias...



Uso básico

2D: 1613 descriptores · 3D: 213 descriptores (requieren conformero)

```
from mordred import Calculator, descriptors
from rdkit import Chem

# Crear calculadora con todos los descriptores 2D
calc = Calculator(descriptors, ignore_3D=True)

# Calcular para una molécula
mol = Chem.MolFromSmiles('CC(=O)Oc1ccccc1C(=O)O')
result = calc(mol)
print(result['MW'])      # → 180.04
print(result['nAcid'])   # → 1
```



Ventajas

- Open source — gratuito
- 1826 descriptores en una línea
- Integración directa con RDKit
- Maneja errores por molécula
- Activamente mantenido



Limitaciones

- Muchos descriptores correlacionados
- 3D requiere conformero previo
- Algunos descriptores sin interp.
- Más lento que RDKit básico
- Necesita selección de features

Dragon · El estándar industrial de descriptores moleculares

Software comercial desarrollado por Talete S.r.l. (Italia). Durante 20 años fue la herramienta de referencia en QSAR industrial y regulatorio.



Características de Dragon 7

5270+ descriptores en 30 bloques temáticos

Bloques principales:

- Constitucionales, topológicos, geométricos
- 3D-MoRSE, WHIM, GETAWAY (espaciales)
- Fingerprints (MACCS, PubChem, Morgan)
- Autocorrelación, potencial de pares
- Descriptores de superficie (CPSA, VSA)

Versiones: Dragon 5, 6 y 7 (actual)

Interfaz gráfica + línea de comandos

Batch processing para miles de moléculas

Usado en modelos QSAR para registro regulatorio (FDA, EMA) — sus descriptores son aceptados en documentación oficial.

TALETE SRL

Dragon 7 available now

The new version of Dragon (7.0) is now available and distributed by Kode srl.

Please visit **Kode website** for details about the new release and on-line order.

Dragon vs Mordred

	Dragon 7	Mordred
Descriptores	5270+	1826
Precio	💰 Comercial	🆓 Gratuito
Python nativo	❌ No	✅ Sí
Validación reg.	✅ Sí (FDA/EMA)	⚠️ No oficial



¿Cómo acceder a Dragon?

Licencia académica: disponible para grupos de investigación universitarios — contactar a Talete (talete.mi.it)

Alternativa práctica: exportar los descriptores desde Dragon y cargarlos en Python con pandas:

En el curso usamos Mordred porque:

- Es gratuito y reproducible
- Se integra directamente en el pipeline
- Sus descriptores son comparables
- Suficiente para modelos QSAR académicos

Selección de Features · Eliminar redundancia e información nula

Con 1826 descriptores, muchos miden cosas similares. Un modelo entrenado con features redundantes es más lento, más propenso al sobreajuste y más difícil de interpretar.

Paso 1 Varianza cero

Descriptores constantes — el mismo valor para todas las moléculas. No aportan información al modelo.

```
VarianceThreshold(threshold=0.01)
# librería: sklearn
```

¿Cuándo usarlo?

Siempre — primer paso obligatorio



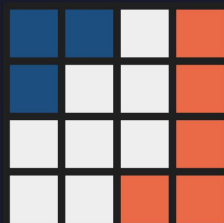
Paso 2 Correlación de Pearson

Para dos descriptores continuos, $r > 0.9$ indica redundancia lineal. Eliminar uno de los dos.

```
corr_matrix = df.corr(method='pearson')
# librería: pandas
```

¿Cuándo usarlo?

Datos continuos, distribuidos normalmente



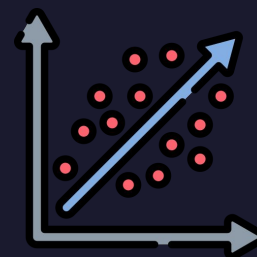
Paso 3 Spearman / Kendall

Si los datos no son normales o tienen outliers, usar correlación de rango (no paramétrica).

```
df.corr(method='spearman')
# librería: pandas
```

¿Cuándo usarlo?

Datos con outliers o distribuciones asimétricas



Paso 4 Imputación de NaN

Algunos descriptores fallan en ciertas moléculas. Columnas con $>20\%$ NaN → eliminar.

```
df.dropna(axis=1, thresh=0.8*len(df))
# librería: pandas
```

¿Cuándo usarlo?

Siempre — antes de cualquier análisis



Regla práctica: umbral $|r| > 0.9 \rightarrow$ redundante (conservar el de mayor varianza) · $|r| < 0.3 \rightarrow$ independientes · $0.3-0.9 \rightarrow$ correlación moderada (mantener ambos)

Reducción de Dimensionalidad · El Espacio Químico

Con 1826 descriptores cada molécula es un punto en un espacio de 1826 dimensiones. La reducción de dimensionalidad proyecta ese espacio a 2D o 3D para visualizarlo.

¿Qué es el espacio químico?

El espacio químico es la representación matemática de todas las moléculas posibles.

Cada punto = una molécula

Cada dimensión = un descriptor

Distancia entre puntos = similitud estructural

Se estima que contiene $\sim 10^{60}$ moléculas drug-like.

Solo hemos explorado $\sim 10^8$ en laboratorio.

La reducción de dimensionalidad proyecta ese espacio de 1826D $\rightarrow$ 2D para que puedas:

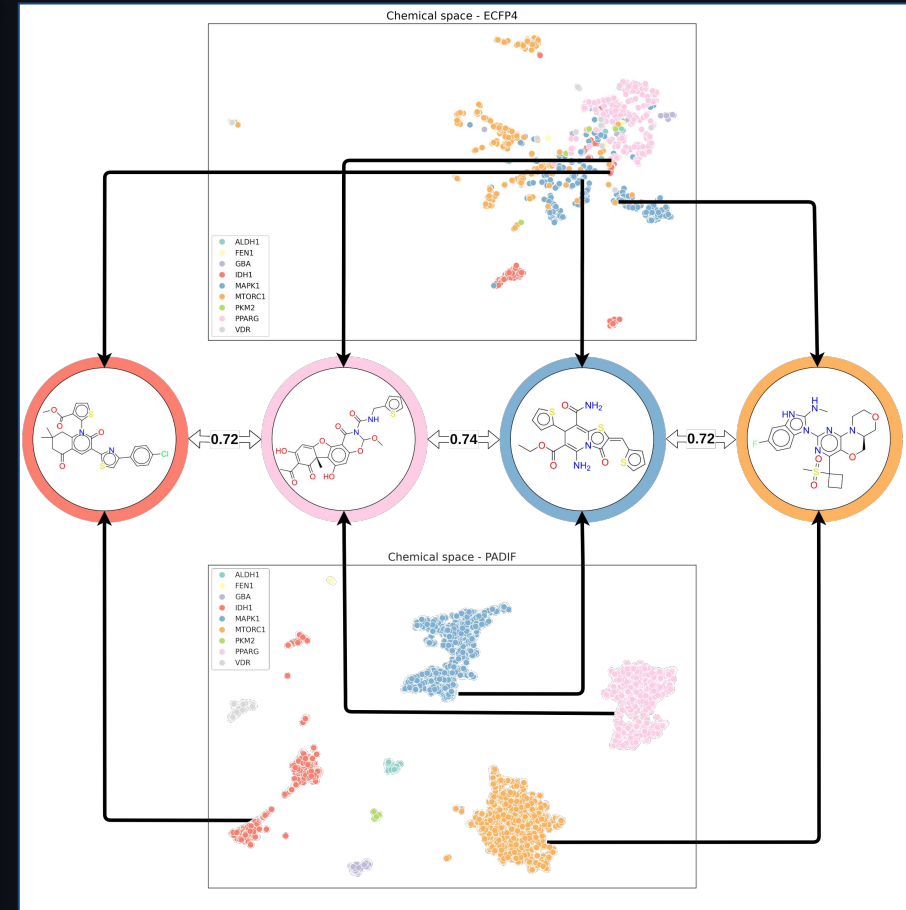
- Ver clusters de familias estructurales
- Identificar zonas inexploradas
- Comparar distribuciones activo/inactivo

La maldición de la dimensionalidad

En alta dimensión ocurren cosas contraintuitivas:

- Todos los puntos se vuelven equidistantes $\rightarrow$ la similitud pierde significado
- Necesitas exponencialmente más datos para cubrir el espacio
- Los modelos memorizan en vez de aprender $\rightarrow$ sobreajuste

Solución: reducir a las dimensiones que concentran la mayor varianza o estructura $\rightarrow$ PCA, UMAP, t-SNE



PCA · Análisis de Componentes Principales

PCA es una transformación lineal que proyecta los datos en las direcciones de máxima varianza. Es el punto de partida obligatorio antes de t-SNE o UMAP.

¿Qué hace PCA?

1. Centra los datos (media = 0)
2. Calcula la matriz de covarianza
3. Obtiene eigenvectores y eigenvalues
4. Ordena por eigenvalue decreciente
5. Proyecta sobre los top-k eigenvectores

Cada componente principal (PC) es una combinación lineal de los descriptores originales, ortogonal a las demás.

PC1 = dirección de máxima varianza

PC2 = dirección ortogonal a PC1
con segunda máxima varianza

... y así sucesivamente

Los datos originales NO se destruyen —
PCA es reversible (con pérdida mínima).

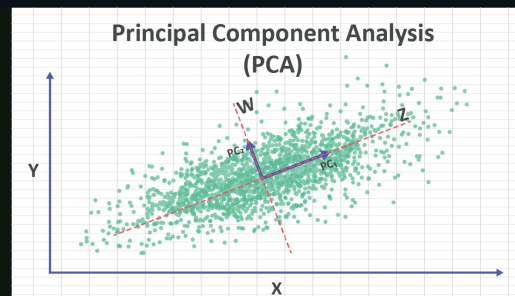
★ Revisar los eigenvalues (scree plot)

El eigenvalue de cada PC indica cuánta varianza explica esa componente.

Regla práctica:

- Buscar el 'codo' en el scree plot
- Quedarte con PCs hasta acumular el 80–90% de la varianza total

Si PC1+PC2 explican < 30% de la varianza, el gráfico 2D puede ser engañoso — considerar usar 3D o t-SNE/UMAP.



INB-DATA-03 · Features y Espacio Químico · UNAL 2026

✓ Ventajas

Determinista — siempre igual

Interpretable (loadings)

Muy rápido

Reducción reversible

Escala bien a millones de muestras

⚠ Limitaciones

Solo captura relaciones lineales

Puede perder estructura no lineal

Los PCs no son descriptores reales

Sensible a la escala → escalar antes

Mal para clusters no convexos

t-SNE y UMAP · Visualización no supervisada del espacio químico

Ambos son métodos de reducción no lineal y no supervisada: aprenden la estructura de los datos sin usar etiquetas. El color que añadimos (activo/inactivo) es solo para interpretación.

¿Por qué son modelos NO supervisados?

No usan las etiquetas (activo/inactivo) para aprender la proyección. Solo ven los descriptores X. Si los activos aparecen agrupados, es porque sus estructuras son similares — no porque el modelo lo sabe.

t-SNE · t-distributed Stochastic Neighbor Embedding

Convierte distancias en probabilidades de vecindad. En alta dimensión usa Gaussianas; en 2D usa distribución t (colas más pesadas). Minimiza la divergencia KL entre ambas.

Hiperparámetros clave:

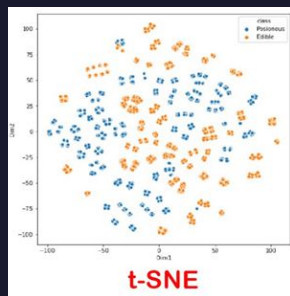
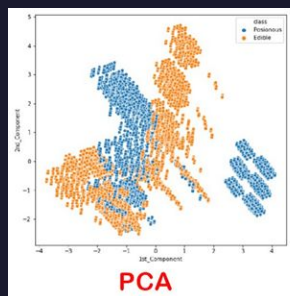
perplexity (5–50): balance local/global

n_iter (≥ 1000): convergencia

metric: 'euclidean', 'cosine', 'jaccard'

⚠ Las distancias absolutas NO tienen significado — solo si los puntos están cerca o lejos entre sí.

⚠ Resultados distintos en cada ejecución
→ usar random_state fijo.



→ Lento para $n > 10.000$ muestras

UMAP · Uniform Manifold Approximation and Projection

Construye un grafo de vecindad en alta dimensión y lo optimiza para preservarlo en 2D. Basado en geometría Riemanniana. Preserva mejor la estructura global.

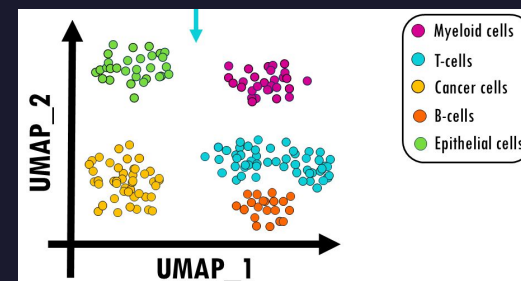
Hiperparámetros clave:

n_neighbors (5–50): vecindad local

min_dist (0–1): compacidad del embedding

metric: 'euclidean', 'cosine', 'jaccard'

✓ Mucho más rápido que t-SNE
✓ Escala a cientos de miles de muestras
✓ Más consistente entre ejecuciones
✓ Las distancias globales SÍ tienen algo de significado (más que en t-SNE)



→ Rápido incluso para $n > 100.000$

Como seleccionar la representación

Todo depende de para que lo necesites: Visualización / Reducción de características

Métodos de reducción de dimensionalidad — comparativa rápida

PCA

Tipo: Lineal
Preserva: Varianza global
⚡ Muy rápido

t-SNE

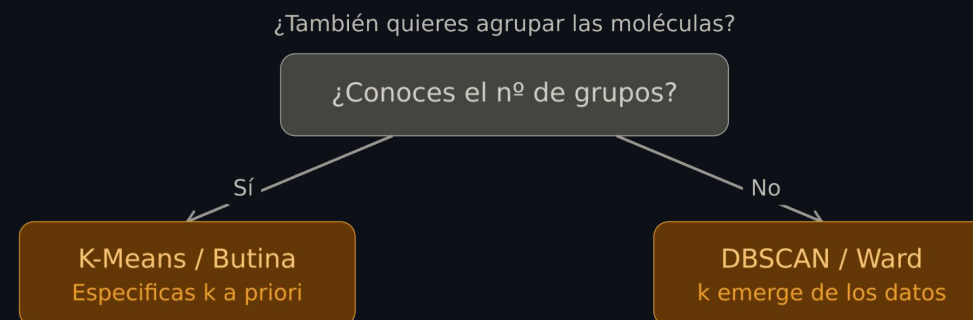
Tipo: No lineal
Preserva: Estructura local
🐢 Lento

UMAP

Tipo: No lineal
Preserva: Local + global
⚡ Rápido

Autoencoder

Tipo: No lineal
Preserva: Reconstrucción
🐢 Lento



Clustering · Agrupar moléculas por similitud estructural

El clustering agrupa automáticamente moléculas similares sin usar etiquetas — es aprendizaje no supervisado. En química medicinal ayuda a entender la diversidad del dataset y a seleccionar representantes.

¿Qué hace el clustering?

Asigna cada molécula a un grupo (cluster) de forma que las moléculas del mismo grupo son más similares entre sí que con las de otros grupos.

La similitud se mide con:

- Descriptores fisicoquímicos (distancia euclídea)
- Fingerprints Morgan (Tanimoto)
- Coordenadas PCA o UMAP (embeddings)

Aplicaciones en drug discovery



Identificar familias de scaffolds -> Agrupar por núcleo estructural común



Selección de conjunto de entrenamiento -> Elegir representantes diversos evita sesgo



Separación train/test scaffold-split -> Test set con scaffolds distintos al train
→ evaluación más realista del modelo



Visualizar el espacio químico -> Color por cluster sobre PCA o UMAP

K-Means

Divide en k clusters predefinidos minimizando la inercia interna. Requiere elegir k.

```
KMeans(n_clusters=8)
```

DBSCAN

Encuentra clusters de densidad variable. Detecta outliers. No requiere k a priori.

```
DBSCAN(eps=0.5, min_samples=5)
```

Butina (Taylor-Butina)

Estándar en quimioinformática. Basado en Tanimoto sobre fingerprints. Sin k a priori.

```
# rdkit.ML.Cluster.Butina
```

Jerárquico (Ward)

Construye un dendrograma. Permite elegir k después. Costoso en memoria ($O(n^2)$).

```
AgglomerativeClustering()
```

En ML: clusters → feature adicional (cluster_id) · selección de muestras representativas · detección de outliers antes del entrenamiento · evaluar diversidad del dataset